

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and
Commerce **Department of Information Technology (B.Sc.I.T**
Semester V)

AI

Practical – VII

Roll No.:T023	Name: Som Mishra
Class:TYIT	Batch:1
Date of Assignment:1-08-2026	Date/Time of Submission:1-08-2026

1. Using any small dataset (e.g., Iris or a CSV dataset), write a Python program to demonstrate the basic machine learning workflow.

a. Load dataset using Pandas.

Code:-

```
import pandas
import pandas as pd

df = pd.read_excel("/content/iris - iris.xlsx")
print(df.head())
```

Output:-

```
...      sepal_length  sepal_width  petal_length  petal_width  species
0           5.1           3.5           1.4           0.2  setosa
1           4.9           3.0           1.4           0.2  setosa
2           4.7           3.2           1.3           0.2  setosa
3           4.6           3.1           1.5           0.2  setosa
4           5.0           3.6           1.4           0.2  setosa
```

b. Perform basic preprocessing (handling missing values or scaling).

```
from sklearn.impute import SimpleImputer
```

```
# Separate features (X)
X = df.iloc[:, :-1]

# Replace missing values with the mean of each column
imputer = SimpleImputer(strategy='mean')
X = imputer.fit_transform(X)
```

Output:-

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

c. Split dataset into training and testing sets.

```
from sklearn.model_selection import train_test_split

# X = Features, y = Target
X = df.iloc[:, :-1]
y = df.iloc[:, -1]

# Split the dataset (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

Output:-

```
• Training data: (120, 4)
  Testing data: (30, 4)
```

d. Train a simple classifier.

Code:-

```
from sklearn.tree import DecisionTreeClassifier

# Create the model
model = DecisionTreeClassifier()

# Train the model
model.fit(X_train, y_train)

print("Classifier trained successfully!")
```

Output:-

```
.. Classifier trained successfully!
```

e. Display training and testing accuracy.

```
Code:-from sklearn.metrics import accuracy_score
from sklearn.metrics import accuracy_score

# Predictions
y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)

# Accuracy
print("Training Accuracy:", accuracy_score(y_train, y_train_pred))
print("Testing Accuracy:", accuracy_score(y_test, y_test_pred))
```

Output:-

```
• Training Accuracy: 1.0
  Testing Accuracy: 1.0
```

Part 2:-

1. Using any small dataset (e.g., Iris or a CSV dataset), write a Python program to demonstrate the basic machine learning workflow.

a. Load dataset using Pandas.

Code:-

```
import pandas
import pandas as pd

df = pd.read_csv("/content/BigMart_Sales.csv")

print(df.head())
```

```
...   Item_Identifier  Item_Weight  Item_Fat_Content  Item_Visibility \
0      FDA15          9.30          Low Fat          0.016047
1      DRC01          5.92          Regular          0.019278
2      FDN15         17.50          Low Fat          0.016760
3      FDX07         19.20          Regular          0.000000
4      NCD19          8.93          Low Fat          0.000000

      Item_Type  Item_MRP  Outlet_Identifier \
0      Dairy    249.8092          OUT049
1  Soft Drinks    48.2692          OUT018
2      Meat    141.6180          OUT049
3  Fruits and Vegetables  182.0950          OUT010
4      Household    53.8614          OUT013

      Outlet_Establishment_Year  Outlet_Size  Outlet_Location_Type \
0              1999          Medium          Tier 1
1              2009          Medium          Tier 3
2              1999          Medium          Tier 1
3              1998              NaN          Tier 3
4              1987          High          Tier 3

      Outlet_Type  Item_Outlet_Sales
0  Supermarket Type1          3735.1380
```

b. Perform basic preprocessing (handling missing values or scaling).

Code:-

```
from sklearn.impute import SimpleImputer

# Separate features (X)
X = df.iloc[:, :-1]

# Handle missing values in numerical columns
num_cols = X.select_dtypes(include=['int64', 'float64']).columns

imputer = SimpleImputer(strategy='mean')
X[num_cols] = imputer.fit_transform(X[num_cols])

print("Missing values handled successfully!")
```

... Missing values handled successfully!

c. Split dataset into training and testing sets.

Code:-

```
from sklearn.model_selection import train_test_split

# Features and Target
X = df.iloc[:, :-1]
y = df.iloc[:, -1]

# Split data (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

Training data: (6818, 11)
Testing data: (1705, 11)

d. Train a simple classifier.

Code:-

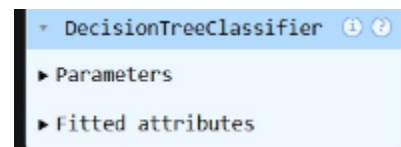
```
from sklearn.tree import DecisionTreeClassifier

classifier = DecisionTreeClassifier(random_state=42)

classifier.fit(X_train, y_train)

✓ 0.0s
```

Output:-



e. Display training and testing accuracy.

Code:-

```
from sklearn.metrics import accuracy_score
```

```
# Predict on training and testing data
train_pred = classifier.predict(X_train)
test_pred = classifier.predict(X_test)

# Calculate accuracy
train_accuracy = accuracy_score(y_train, train_pred)
test_accuracy = accuracy_score(y_test, test_pred)

# Display accuracy
print("Training Accuracy :", round(train_accuracy * 100, 2), "%")
print("Testing Accuracy :", round(test_accuracy * 100, 2), "%")
```

Output:-

```
Training Accuracy : 100.0 %
Testing Accuracy : 75.31 %
```

Q3. Compare Training Accuracy and Testing Accuracy

Point	Training Accuracy	Testing Accuracy
Definition	Accuracy measured on the training dataset.	Accuracy measured on the testing dataset.
Purpose	Evaluates how well the model learns the training data.	Evaluates how well the model performs on unseen data.
Dataset Used	Training dataset (X_train, y_train)	Testing dataset (X_test, y_test)
Prediction	model.predict(X_train)	model.predict(X_test)
Formula	accuracy_score(y_train, train_pred)	accuracy_score(y_test, test_pred)
Expected Value	Usually higher.	Usually slightly lower.
Indicates	Learning performance of the model.	Generalization performance of the model.

**Example
Result**

100.00%

87.45%

This is the type of **comparison table** commonly accepted in practical files and viva examinations.